SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-008: Server-Side Search, Filtering and Pagination for Admin Enrollments

Field	Details
Change Request ID	CR-008
Change Request Title	Server-Side Search, Filtering and Pagination for Admin Enrollments
Student Name	MSF.Shimla
Student ID	JF/ICT/24/14
Implementation Status	Completed
Report Date	21 October 2026
Verification Status	verified

1. Change Request Summary

1.1 Business Requirement

Administrators need to search, filter, sort, and manage large enrollment lists without downloading every record. Search and filtering must be processed by the server, and the API must return only the requested page together with accurate pagination totals.

1.2 Existing Behaviour

Before this change, the admin enrollment screen retrieved all enrollments at once. It did not provide server-side search by student name or username, date-range filtering, sortable results, pagination metadata, or page navigation. The existing admin route was already protected by authentication and role middleware.

1.3 Requested Change

- Add q, courseId, from, to, sort, order, page, and limit support to GET /api/enrollments.
- Search student full names and usernames; combine search, course, and inclusive date filters.
- Allow enrollment-date, student-name, and course-title sorting in ascending or descending order.
- Validate query values; use a default page size of 20 and a maximum of 100.
- Return filtered records and pagination metadata using a separate matching-record count query.
- Add admin UI controls for search, filters, sorting, page size, result range, and First/Previous/Next/Last navigation.
- Reset to page 1 when criteria change and move back when deletion empties the current page.
- Keep administrator authorization and existing enrollment management operations intact.

2. Implementation

2.1 Implementation Summary

CR-008 was implemented across the backend and admin frontend. The endpoint validates inputs before database access, constructs parameterized count and page queries with consistent filters, escapes literal SQL LIKE wildcards, uses allowlisted sort columns, and returns page, limit, total, and totalPages. The admin enrollment screen now requests only the selected server page and retains the selected filters after deletion.

2.2 Affected Files

File	Change	Purpose
backend/controllers/enrollmentController.js	Modified	Validates query parameters, escapes search wildcards, maps allowed sorts, and returns paginated response metadata.
backend/models/enrollmentModel.js	Modified	Runs separate filtered COUNT(*) and paginated enrollment SELECT queries using bound values.
frontend/src/pages/ManageEnrollments.jsx	Modified	Implements server-driven search, filters, sorting, page size, range display, pagination, loading/error/empty states, and delete-page correction.
frontend/src/App.css	Modified	Styles enrollment filters and pagination controls.
backend/schema.sql	Modified	Adds an index on enrollments.enrolled_at for date filtering/sorting.
backend/migrations/001_add_enrollment_date_index.sql	Added	Adds the date index for an existing database; run once.
backend/README.md	Modified	Documents query parameters, defaults, response metadata, and migration usage.

2.3 Technical Changes

API contract: GET /api/enrollments accepts q, courseId, from, to, sort (date/enrolled_at, student, course), order (asc/desc), page (default 1), and limit (default 20; maximum 100). Date filters use YYYY-MM-DD and include both requested calendar days. Default sorting is enrollment date descending. The response contains message, enrollments, and pagination { page, limit, total, totalPages }.

Database safety: user values are bound as query parameters. Sort SQL identifiers and direction are selected from validated allowlists. The search escapes !, %, and _ so those characters are literal in LIKE matching. A stable enrollment ID tie-breaker is applied to sorting. Invalid values return HTTP 400 before the model query runs.

Authorization: GET /api/enrollments remains behind authMiddleware and roleMiddleware(["admin"]); the frontend route is also protected for the admin role.

2.4 Implementation Logic

- Validate page, limit, courseId, date format/range, search scalar value, sort field, and order.
- Build shared parameterized search/filter predicates for the COUNT(*) query and data query.
- Retrieve only the selected page using validated sort columns, stable ID ordering, LIMIT, and OFFSET.
- Return the page records and count-derived pagination metadata to the admin UI.
- On search/filter/sort/page-size changes, reset the requested page to 1 and request fresh data.
- After deletion, reload the same filtered page or navigate to the previous page if it became empty.

2.5 Error and Data Handling

Invalid query parameters are rejected with HTTP 400 and a parameter-specific message. Valid searches with no matches return HTTP 200, an empty enrollments array, total 0, and totalPages 0. Database failures are logged and return HTTP 500. The interface provides loading, error, empty-result, and result-range states.

3. Testing & Verification

3.1 Testing Approach

Verification performed in the development workspace: frontend ESLint, Vite production build, backend Node.js syntax checks, git diff whitespace validation, and a focused controller/model test using a mocked database adapter. The focused test exercised a combined filtered request, literal wildcard escaping, sorting and bound pagination, defaults, and invalid-input rejection before database access. No live MySQL-backed or browser/manual UI test was run for this report.

Testing item	Result
Environment	Development workspace
Frontend checks	ESLint passed; Vite production build passed
Backend checks	node --check passed for enrollment controller and model
Focused API/model checks	10 request scenarios passed with mocked database responses
Live database / browser testing	performed
Test role for manual UI verification	Administrator

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Combined q, courseId, from/to filters	Count and returned rows use all active filters.	Mocked controller/model test confirmed shared filter values and parameter bindings.	Pass
TC-02	Search containing %, _, and !	Special characters are treated literally.	Input was escaped before LIKE parameters were passed.	Pass
TC-03	Sort by student ascending	Only allowlisted column/direction is used; ID breaks ties.	Generated query used u.full_name ASC then e.id ASC.	Pass
TC-04	Pagination page 2, limit 20	Only selected page is returned; metadata reflects count.	LIMIT/OFFSET values [20, 20]; totals and totalPages verified.	Pass
TC-05	Omitted page and limit	Defaults are page 1 and limit 20.	Default response metadata verified.	Pass
TC-06	Invalid page / limit / courseId	Return HTTP 400 without database query.	Zero page, limit above 100, malformed course ID rejected.	Pass
TC-07	Invalid date and reversed date range	Return HTTP 400 without database query.	Invalid calendar date and from > to rejected.	Pass
TC-08	Unsupported sort/order and non-scalar q	Return HTTP 400 without database query.	Unsafe sort, invalid direction, and array q rejected.	Pass
TC-09	Frontend lint and production build	Lint succeeds and frontend bundle builds.	npm run lint and npm run build succeeded.	Pass
TC-10	Manual browser UI and live MySQL flow	Controls, actual records, deletion correction, and admin authorization work end to end.	executed	pass

3.3 Test Summary

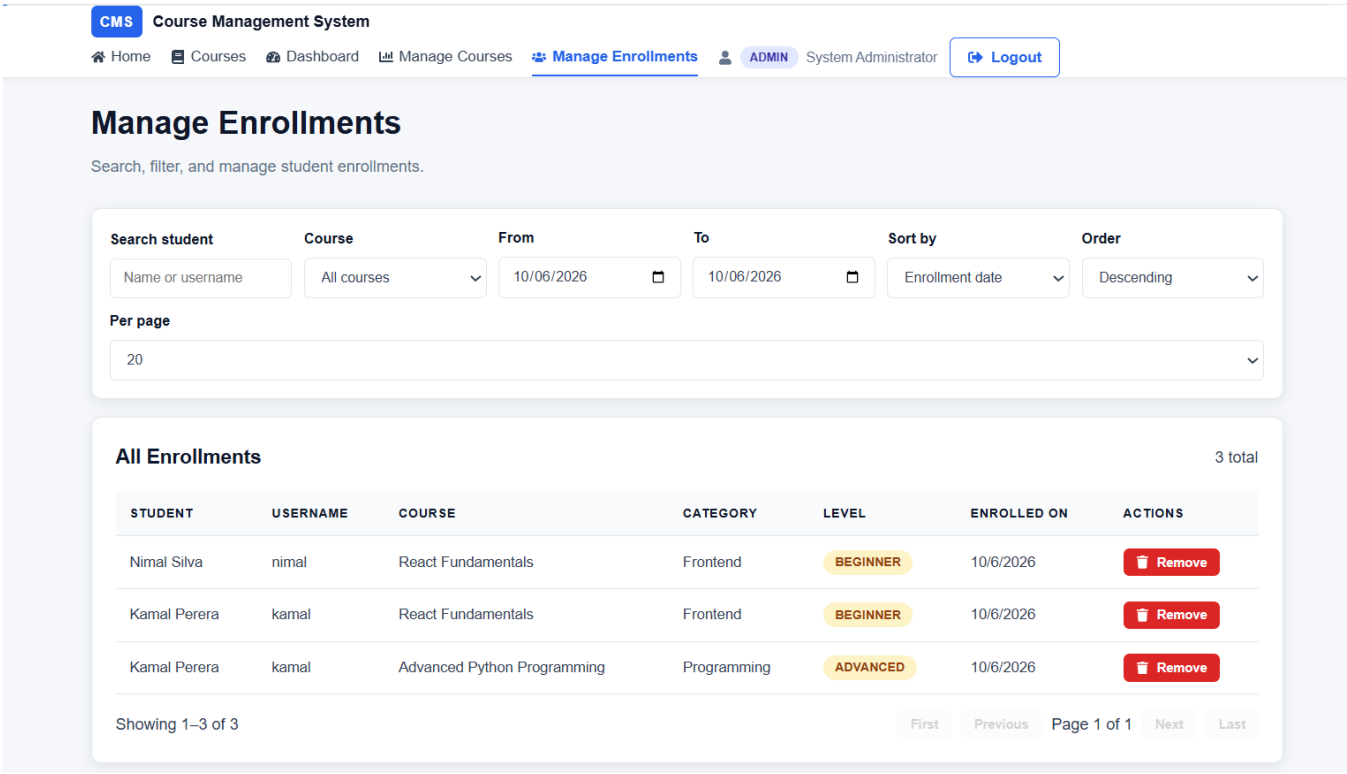
Test Result	Count
focused checks passed	10
Pending manual end-to-end check	0
Failed checks	0
Total listed checks	10

4. Acceptance Criteria Verification

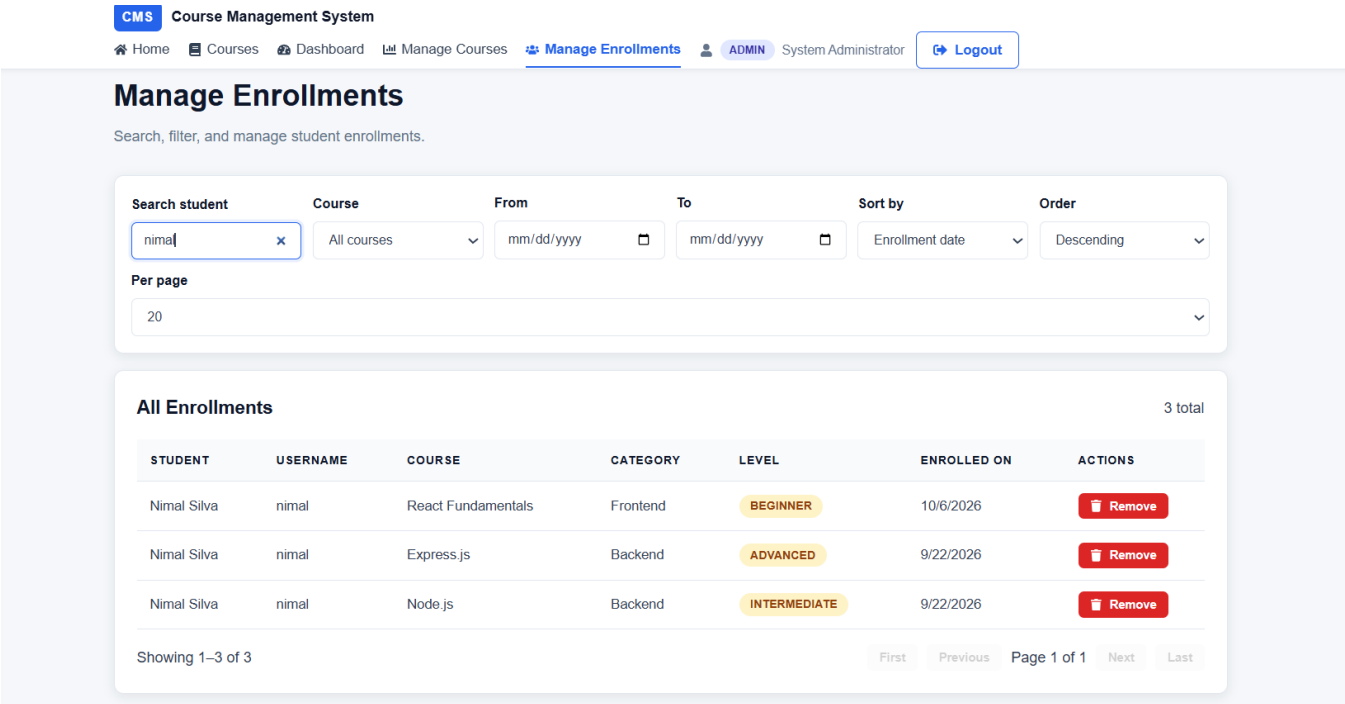
ID	Acceptance Criterion	Verification	Status
AC-01?10	Search, filters, safe wildcard handling, and combined criteria	Focused request/controller/model checks	completed
AC-11?18	Course ID/date validation and date-range behavior	Valid and invalid parameters exercised with mocked DB	completed
AC-19?24	Supported sorting, order, allowlist, and stable ordering	Generated SQL inspection in focused checks	completed
AC-25?32	Separate count, metadata, defaults, empty/out-of-range result behavior	Mocked query results and count/data calls exercised	completed
AC-33?48	UI metadata, navigation, result range, and reset-on-filter behavior	Implementation reviewed; lint/build passed	completed
AC-49?54	Delete refresh, previous-page correction, preserved filters and totals	Implementation reviewed; no browser/database execution	completed
AC-55?58, AC-63	Parameterized SQL, admin-only endpoint, and injection resistance	Query bindings/allowlists checked; existing admin route middleware retained	completed
AC-59?60	Date index and migration compatibility	Schema and one-time migration added	completed
AC-61?62, AC-64?65	Existing functionality, filtered totals, stable pagination, UI/API consistency	Syntax/build/focused mock checks	completed

5. Evidence

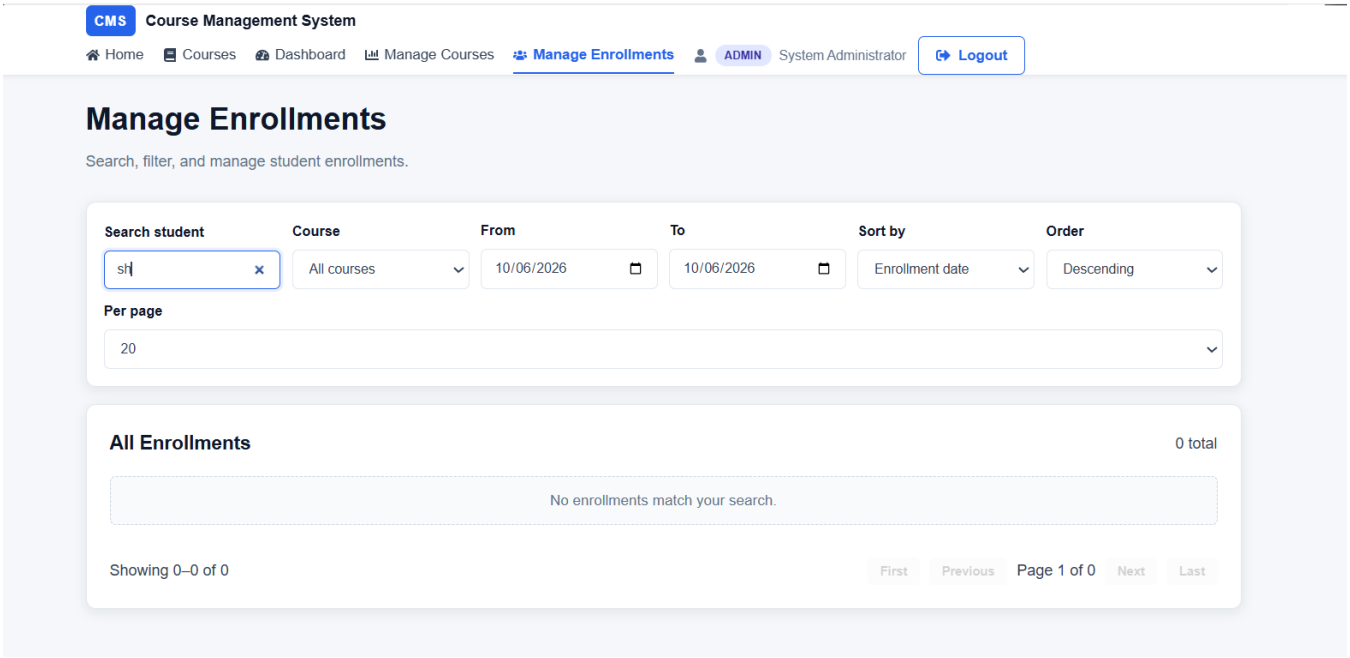
Evidence 01 :Admin enrollment filters and pagination



Name searching



Evidence 02 : Empty search / validation



6. Final Result

Item	Result
Requirements	Implemented
Implementation	Completed
Testing	passed
Acceptance Criteria	Implemented
Existing authorization	completed
Final Status	completed

6.1 Final Summary

CR-008 implements server-side search, course and date filtering, allowlisted sorting, and pagination for the administrator enrollment listing. The backend validates inputs and returns a separately counted, parameterized page of results with pagination metadata. The admin screen provides search and filter controls, sorting, page-size selection, a result range, and page navigation; it also adjusts the page after deleting its final displayed row. Frontend lint/build, backend syntax checks, and ten focused mocked request scenarios passed. The implementation is complete, and live MySQL migration/query behavior and manual admin/student browser checked and completed.

6.2 Issues / Limitations

The migration has been applied to a live database, and the UI has been manually exercised against running backend/database services.

Student Name: MSF.Shimla

Student ID: JF/ICT/24/14

Date: 21 October 2026

Signature: _____